

Доклад – Използване на Async операции в библиотечната система

1. С какво спомага Async в проекта

В проекта асинхронните операции се използват при работа с базата данни чрез Entity Framework Core. Те позволяват приложението да обработва заявки без да блокира основния поток на изпълнение.

Това подобрява производителността на приложението и позволява на сървъра да обработва повече потребители едновременно. Докато се изпълнява заявка към базата данни, приложението може да обработва и други заявки.

2. Каква заявка към базата данни се изпълнява

В проекта се използва заявка за извличане на книги от базата данни.

Заявката работи с модела Book и извлича всички книги заедно с информация за техните автори.

Използваният код е:

```
var applicationDbContext = _context.Books.Include(b => b.Author);  
return View(await applicationDbContext.ToListAsync());
```

Операцията извършва извличане на данни от таблиците Books и Authors.

3. В кой контролер се намира

Заявката се намира в:

- контролер: BooksController
- action метод: Index()

Методът е дефиниран като асинхронен чрез ключовите думи async и await.

4. Синхронен и асинхронен вариант

Синхронен вариант

```
public IActionResult Index()
{
    var books = _context.Books.Include(b => b.Author).ToList();

    return View(books);
}
```

При синхронното изпълнение приложението изчаква заявката към базата данни да приключи, преди да продължи изпълнението. По време на изчакването потокът остава блокиран.

Асинхронен вариант

```
public async Task<IActionResult> Index()
{
    var books = await _context.Books
        .Include(b => b.Author)
        .ToListAsync();

    return View(books);
}
```

При асинхронното изпълнение заявката се изпълнява без блокиране на основния поток. Докато базата данни обработва заявката, сървърът може да изпълнява и други операции.

5. Как Async подобрява приложението

Async операциите подобряват производителността на приложението, особено при работа с база данни и повече потребители.

При синхронните операции сървърът изчаква всяка заявка да приключи, което може да доведе до забавяне. При асинхронните операции потокът се освобождава и може да обработва други заявки.

Това позволява:

- по-бърза работа на приложението;
- по-добро използване на ресурсите;
- обработка на повече потребители едновременно;
- по-малко натоварване на сървъра.

Използването на `async` и `await` е важна част от съвременните ASP.NET Core приложения и подобрява ефективността при работа с база данни.